

UNIVERSIDAD MAYOR DE SAN ANDRÉS
FACULTAD DE CIENCIAS PURAS Y NATURALES
CARRERA DE INFORMÁTICA



PROYECTO FINAL DE CURSO
PROPUESTA DE SOLUCIÓN CON INTELIGENCIA ARTIFICIAL
IMPLEMENTACION DE ALGORITMO MACHINE LEARNING
RANDOM FOREST PROGRESSOR
“DriveSafe IA”

*Sistema Inteligente de Detección de Somnolencia
para Transporte Interdepartamental y Vehículos Particulares*

INTEGRANTES: Univ. López Ticona Angela Magali
Univ. Mendoza Caspa Ronald
Univ. Usnayo Velasco Nathalie Maya
Univ. Zapata Nina Harol Mirko

FECHA: 27 de Julio de 2026

DOCENTE: Lic. Huayna Magne Andrea Belen

ASIGNATURA: Inteligencia Artificial

LA PAZ – BOLIVIA

1. Definición del Dataset

El sistema utiliza un dataset propio, generado automáticamente durante el funcionamiento de la aplicación de monitoreo de fatiga. Cada vez que el sistema registra una alerta, almacena una muestra con las variables necesarias para el entrenamiento del modelo de Machine Learning.

Las muestras se guardan en la tabla **ml_samples** de la base de datos y posteriormente son exportadas al archivo **dataset.csv**, el cual es utilizado por el script **train_model.py** para entrenar el modelo.

El dataset está compuesto por las siguientes variables:

Variable	Tipo	Descripción
ojos	Double	Nivel de apertura de los ojos (EAR).
bostezos	Entero	Cantidad de bostezos detectados.
tiempo	Double	Tiempo de conducción en minutos.
fatiga	Entero	Nivel de fatiga utilizado como variable objetivo para el entrenamiento.

El conjunto de datos se actualiza de forma continua, ya que cada nueva alerta registrada incorpora una nueva muestra al dataset, permitiendo reentrenar el modelo con información más reciente.

2. Análisis Exploratorio de Datos (EDA)

2.1 Descripción del conjunto de datos

El conjunto de datos utilizado por el sistema está compuesto por las variables capturadas durante el monitoreo del conductor. Cada registro corresponde a un evento registrado por la aplicación y posteriormente almacenado en la tabla **ml_samples**, desde donde se exporta el archivo **dataset.csv** para el entrenamiento del modelo de Machine Learning.

Las variables del conjunto de datos son las siguientes:

Variable	Tipo	Descripción
ojos	Numérico (double)	Valor del EAR (Eye Aspect Ratio) obtenido mediante visión artificial. Representa el nivel de apertura de los ojos del conductor.

bostezos	Entero	Cantidad de bostezos detectados durante el monitoreo.
tiempo	Numérico (double)	Tiempo de conducción continuo expresado en minutos desde el inicio del monitoreo.
fatiga	Entero	Nivel de fatiga del conductor expresado en una escala de 0 a 100. Esta variable es utilizada como objetivo del modelo de predicción.

2.2 Tipo de variables

El conjunto de datos está conformado únicamente por variables numéricas, lo que permite entrenar directamente un modelo de regresión sin requerir codificación de variables categóricas.

Variable	Tipo de dato	Rol
ojos	Numérica continua	Variable de entrada
bostezos	Numérica discreta	Variable de entrada
tiempo	Numérica continua	Variable de entrada
fatiga	Numérica discreta	Variable objetivo

2.3 Distribución de los datos

El comportamiento esperado de las variables es el siguiente:

- **ojos**: valores bajos indican mayor tiempo con los ojos cerrados y, por tanto, mayor probabilidad de fatiga.
- **bostezos**: conforme aumenta la cantidad de bostezos también aumenta la probabilidad de fatiga.
- **tiempo**: mientras mayor sea el tiempo continuo de conducción, mayor es el riesgo de fatiga.
- **fatiga**: representa el nivel estimado por el sistema en una escala de 0 a 100.

Estas variables presentan una relación directa con el estado de somnolencia del conductor y constituyen las características utilizadas durante el entrenamiento del modelo Random Forest Regressor.

2.4 Valores faltantes

Durante el proceso de exportación del conjunto de datos únicamente se consideran los registros completos.

La consulta utilizada por el sistema es:

```
SELECT ojos, bostezos, tiempo, fatiga
FROM ml_samples
WHERE ojos IS NOT NULL
AND bostezos IS NOT NULL
AND tiempo IS NOT NULL
AND fatiga IS NOT NULL;
```

Gracias a este filtro, el archivo dataset.csv contiene únicamente registros completos, evitando problemas derivados de valores faltantes durante el entrenamiento.

2.5 Correlación entre variables

De acuerdo con la lógica implementada en el sistema, las tres variables de entrada presentan una relación con la variable objetivo (fatiga):

- Una disminución del valor **EAR (ojos)** incrementa el nivel de fatiga.
- Un aumento en el número de bostezos incrementa el nivel de fatiga.
- Un mayor tiempo de conducción incrementa el nivel de fatiga.

Estas tres variables fueron seleccionadas por representar indicadores fisiológicos asociados al estado de somnolencia del conductor y constituyen los atributos utilizados por el modelo de Machine Learning para realizar la predicción.

3. Preprocesamiento y Limpieza de Datos

3.1 Limpieza de datos

Las muestras utilizadas por el modelo de aprendizaje automático provienen de la tabla **ml_samples**, donde se almacenan los datos capturados durante el monitoreo del conductor. Antes del entrenamiento del modelo, únicamente se seleccionan los registros que contienen información completa para las variables necesarias.

En el archivo **ml_admin.php** se ejecuta la siguiente consulta:

```
SELECT ojos, bostezos, tiempo, fatiga
FROM ml_samples
WHERE ojos IS NOT NULL
AND bostezos IS NOT NULL
AND tiempo IS NOT NULL
AND fatiga IS NOT NULL;
```

Con esta validación se evita utilizar registros incompletos que puedan afectar el entrenamiento del modelo.

3.2 Tratamiento de valores faltantes

En lugar de aplicar técnicas de imputación, el sistema descarta automáticamente las muestras que presentan valores nulos en cualquiera de las variables principales.

De esta manera, únicamente los registros completos son exportados hacia el archivo **dataset.csv**, garantizando que el algoritmo de aprendizaje automático trabaje con información consistente.

3.3 Normalización de datos

En el presente proyecto no fue necesario aplicar procesos de normalización o escalado de datos.

Las variables utilizadas presentan rangos numéricos relativamente similares:

Variable	Tipo
ojos	Decimal
bostezos	Entero
tiempo	Decimal
fatiga	Entero

Debido a que el algoritmo seleccionado corresponde a Random Forest Regressor, éste no requiere que las variables sean normalizadas para obtener un buen desempeño.

3.4 Codificación de variables

No fue necesario aplicar técnicas de codificación como One-Hot Encoding o Label Encoding, debido a que todas las variables empleadas por el modelo son numéricas.

Las características utilizadas durante el entrenamiento fueron:

- ojos
- bostezos
- tiempo

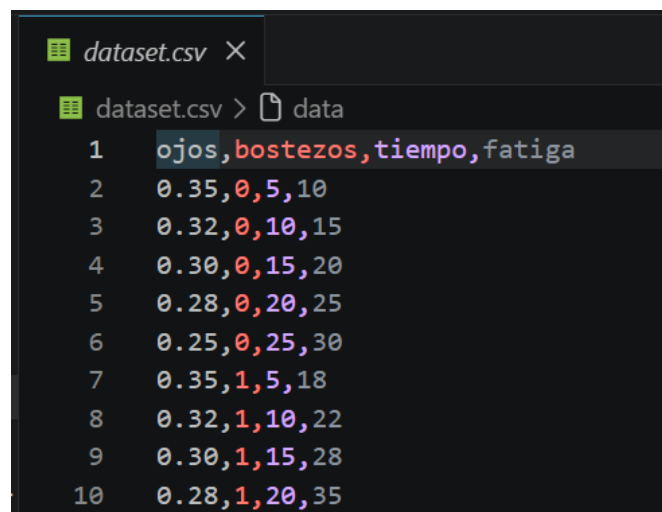
La variable objetivo corresponde a:

- fatiga

3.5 Exportación del conjunto de entrenamiento

Después del filtrado de registros válidos, el sistema genera automáticamente el archivo `dataset.csv`, el cual contiene la información utilizada para entrenar el modelo de Machine Learning.

El archivo posee la siguiente estructura:



```
dataset.csv X
dataset.csv > data
1  ojos,bostezos,tiempo,fatiga
2  0.35,0,5,10
3  0.32,0,10,15
4  0.30,0,15,20
5  0.28,0,20,25
6  0.25,0,25,30
7  0.35,1,5,18
8  0.32,1,10,22
9  0.30,1,15,28
10 0.28,1,20,35
```

Posteriormente este archivo es procesado por el script `train_model.py`, encargado de entrenar el modelo Random Forest y generar el archivo

modelo_fatiga.pkl, el cual será utilizado posteriormente durante la predicción en tiempo real.

4. Ingeniería de Características

4.1 Selección de características

Para el entrenamiento del modelo de Machine Learning se seleccionaron tres variables que representan el comportamiento del conductor durante el monitoreo en tiempo real. Estas variables fueron elegidas porque permiten estimar el nivel de fatiga de forma objetiva.

Las características utilizadas fueron las siguientes:

Característica	Descripción
ojos	Valor numérico obtenido mediante la detección del grado de apertura de los ojos (EAR - Eye Aspect Ratio).
bostezos	Cantidad de bostezos detectados durante la sesión de monitoreo.
tiempo	Tiempo transcurrido desde el inicio del viaje, expresado en minutos.

La variable objetivo utilizada durante el entrenamiento corresponde a:

Variable objetivo	Descripción
fatiga	Nivel de fatiga estimado del conductor, expresado en una escala de 0 a 100.

4.2 Extracción de características

Las características son obtenidas automáticamente desde el módulo de monitoreo desarrollado en JavaScript (**monitor.js**).

Durante el funcionamiento del sistema se realiza el análisis del rostro del conductor mediante la cámara web. A partir de este análisis se calculan las siguientes medidas:

- Nivel de apertura de los ojos.
- Cantidad de bostezos detectados.
- Tiempo de conducción transcurrido.

Estas variables son enviadas al servidor junto con la alerta generada para ser almacenadas en la tabla **ml_samples**, donde posteriormente son utilizadas para el entrenamiento del modelo.

4.3 Transformación de características

No fue necesario aplicar transformaciones adicionales sobre las variables seleccionadas, ya que todas corresponden a valores numéricos obtenidos directamente del sistema de monitoreo.

Las variables son utilizadas por el modelo exactamente en el mismo formato en que fueron capturadas.

4.4 Justificación de las características seleccionadas

Las variables elegidas representan indicadores directamente relacionados con el estado de fatiga del conductor.

- **ojos**: permite identificar periodos prolongados de cierre ocular o microsueños.
- **bostezos**: representa uno de los indicadores fisiológicos más comunes de cansancio.
- **tiempo**: considera el incremento de la fatiga conforme aumenta el tiempo de conducción.

La combinación de estas tres variables proporciona suficiente información para que el algoritmo Random Forest pueda estimar el nivel de fatiga de manera adecuada.

5. Entrenamiento y Evaluación de Modelos

5.1 Entrenamiento del modelo

El modelo de Machine Learning fue desarrollado utilizando la biblioteca **Scikit-learn** en Python. Para el entrenamiento se empleó el algoritmo **Random Forest Regressor**, el cual utiliza múltiples árboles de decisión para realizar predicciones sobre el nivel de fatiga del conductor.

El proceso de entrenamiento se realiza mediante el archivo **train_model.py**, donde se carga el conjunto de datos (**dataset.csv**), se seleccionan las variables de entrada y posteriormente se entrena el modelo.

Las variables utilizadas fueron:

- ojos
- bostezos
- tiempo

La variable objetivo corresponde a:

- fatiga

Una vez finalizado el entrenamiento, el modelo es almacenado en el archivo **modelo_fatiga.pkl**, permitiendo reutilizarlo posteriormente para realizar predicciones en tiempo real sin necesidad de volver a entrenarlo.

5.2 Comparación de algoritmos

Aunque el sistema implementa únicamente **Random Forest Regressor** para la predicción de fatiga, durante el análisis se consideraron otros algoritmos de aprendizaje supervisado que podrían emplearse para este tipo de problema.

Algoritmo	Ventajas	Desventajas
Random Forest Regressor	Buena precisión, tolera ruido en los datos y reduce el sobreajuste mediante múltiples árboles de decisión.	Mayor tiempo de entrenamiento y mayor consumo de memoria.
Regresión Lineal	Implementación sencilla, entrenamiento rápido y fácil interpretación.	Presenta menor capacidad para modelar relaciones no lineales entre las variables.

De acuerdo con las características del problema, se seleccionó **Random Forest Regressor**, ya que permite modelar de mejor manera la relación entre las características del conductor y el nivel de fatiga estimado.

5.3 Evaluación del modelo

El modelo fue evaluado mediante pruebas funcionales realizadas durante el desarrollo del sistema.

Durante las pruebas se verificó que, al enviar los valores correspondientes a la apertura de los ojos, cantidad de bostezos y tiempo de conducción, el modelo devuelve un nivel de fatiga comprendido entre 0 y 100.

Esta predicción es utilizada inmediatamente por el sistema para clasificar el nivel de riesgo del conductor y generar las alertas correspondientes.

Debido a que el proyecto se enfoca en la integración del modelo dentro de una aplicación web en tiempo real, no se implementó un módulo específico para el cálculo automático de métricas como Accuracy, F1-Score, MSE o Matriz de Confusión.

5.4 Resultado del entrenamiento

El entrenamiento genera los siguientes archivos:

dataset.csv, que contiene las muestras utilizadas para el entrenamiento.

modelo_fatiga.pkl, correspondiente al modelo entrenado.

Estos archivos permiten mantener actualizado el modelo conforme se recopilan nuevas muestras durante el funcionamiento del sistema.

6. Selección y Exportación del Modelo Final

6.1 Selección del modelo

Para el desarrollo del sistema se seleccionó el algoritmo **Random Forest Regressor** de la biblioteca **Scikit-Learn**. Este modelo fue elegido debido a que presenta un buen desempeño para problemas de regresión y es capaz de modelar relaciones no lineales entre las variables utilizadas para estimar el nivel de fatiga del conductor.

Las variables empleadas para el entrenamiento fueron:

- Apertura de los ojos (EAR).
- Cantidad de bostezos detectados.
- Tiempo de conducción en minutos.

Como resultado del entrenamiento, el modelo aprende la relación existente entre estas variables y el porcentaje de fatiga que posteriormente será utilizado por el sistema para generar alertas automáticas.

6.2 Exportación del modelo

Una vez finalizado el entrenamiento, el modelo es serializado utilizando la biblioteca **Joblib**, permitiendo almacenarlo para su utilización posterior sin necesidad de volver a entrenarlo.

El archivo generado corresponde a:

modelo_fatiga.pkl

El proceso de exportación se realiza mediante la siguiente instrucción que está en `train_model.py`:

`joblib.dump(modelo, "modelo_fatiga.pkl")`

Gracias a este procedimiento, el modelo puede cargarse posteriormente desde diferentes componentes del sistema, reduciendo significativamente el tiempo de respuesta durante la ejecución de las predicciones.

6.3 Versionado del modelo

Cada vez que el administrador ejecuta un nuevo entrenamiento desde el módulo de Machine Learning, el sistema realiza automáticamente las siguientes acciones:

- Exporta las muestras almacenadas en la base de datos hacia el archivo `dataset.csv`.
- Entrena nuevamente el modelo Random Forest.
- Genera una nueva versión de `modelo_fatiga.pkl`.
- Crea una copia de respaldo del modelo anterior.

Este procedimiento permite mantener un historial de versiones del modelo entrenado y facilita su actualización conforme se recolectan nuevas muestras provenientes del sistema de monitoreo.

7. Diseño de la Arquitectura del Sistema

7.1 Arquitectura general

El sistema **DriveSafe AI** fue desarrollado utilizando una arquitectura modular, donde cada componente cumple una función específica dentro del proceso de monitoreo de fatiga del conductor. Esta arquitectura integra tecnologías web, servicios de inteligencia artificial, base de datos y procesamiento en tiempo real para ofrecer una solución completa de detección y prevención de accidentes.

Los principales componentes del sistema son:

- **Frontend Web:** desarrollado en PHP, HTML, CSS y JavaScript, encargado de la interacción con el usuario y de la captura de información desde la cámara y el GPS.
- **Backend:** implementado en PHP, responsable de procesar las solicitudes, administrar usuarios, gestionar rutas, registrar alertas y comunicarse con el servicio de Machine Learning.
- **Servicio de Inteligencia Artificial:** desarrollado en Python utilizando FastAPI y Scikit-Learn, encargado de realizar la predicción del nivel de fatiga mediante el modelo Random Forest entrenado.
- **Base de Datos MySQL:** almacena la información de usuarios, conductores, vehículos, rutas, alertas, posiciones GPS y muestras utilizadas para el entrenamiento del modelo.

Durante el funcionamiento del sistema, las características detectadas por el navegador (apertura de ojos, cantidad de bostezos y tiempo de conducción) son enviadas al servidor. Posteriormente, estas variables son procesadas por el modelo de Machine Learning para estimar el nivel de fatiga del conductor y generar las alertas correspondientes.

7.1.1 Diagrama de Base de Datos

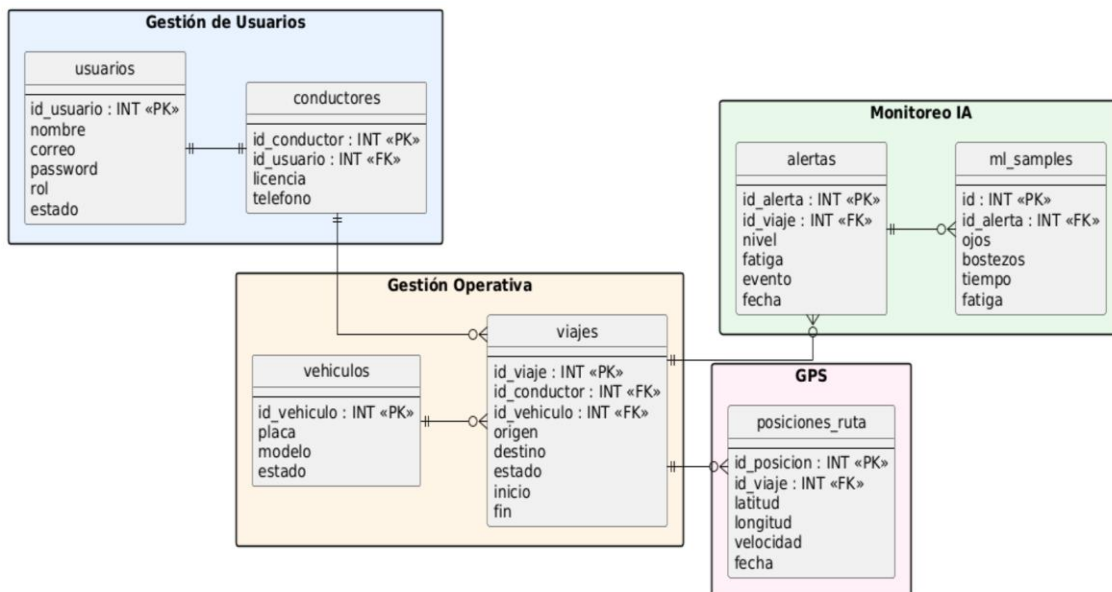
La base de datos fue diseñada para almacenar toda la información necesaria para el funcionamiento del sistema de monitoreo inteligente. Entre las principales entidades se encuentran:

- Usuarios
- Conductores
- Vehículos
- Viajes
- Posiciones de Ruta (GPS)
- Alertas
- Muestras de Machine Learning (ml_samples)

Las relaciones entre estas entidades permiten registrar el historial completo de los recorridos realizados, las alertas generadas y los datos utilizados posteriormente para reentrenar el modelo de inteligencia artificial.

Principales tablas utilizadas por el módulo de IA

Tabla	Descripción
usuarios	Información de administradores, supervisores y conductores.
viajes	Registro de rutas programadas y recorridos realizados.
posiciones_ruta	Historial de coordenadas GPS durante el viaje.
alertas	Eventos de fatiga detectados durante el monitoreo.
ml_samples	Almacena las variables utilizadas para entrenar el modelo (ojos, bostezos, tiempo y nivel de fatiga).



7.1.2 Diagrama de Componentes

La arquitectura del sistema está organizada en cuatro componentes principales que interactúan entre sí para realizar el monitoreo en tiempo real:

1. Frontend Web

- monitor.php
- monitor.js
- MediaPipe Face Mesh
- Geolocalización GPS

2. Backend PHP

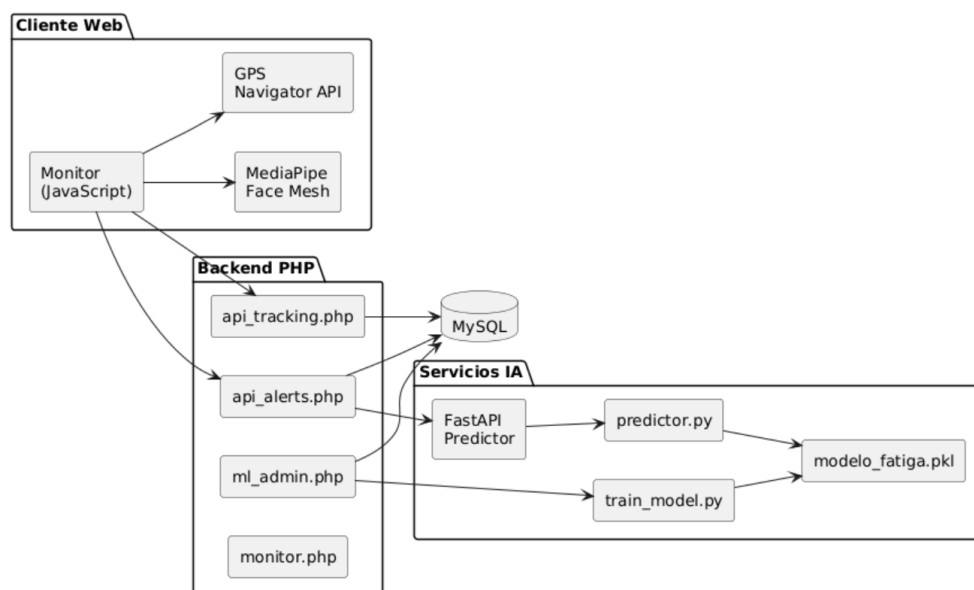
- api_alerts.php
- api_tracking.php
- management.php
- ml_admin.php

3. Servicio de IA

- predictor_service.py
- predictor.py
- modelo_fatiga.pkl

4. Base de Datos

- MySQL
- Tabla ml_samples
- Tabla alertas
- Tabla viajes
- Tabla posiciones_ruta



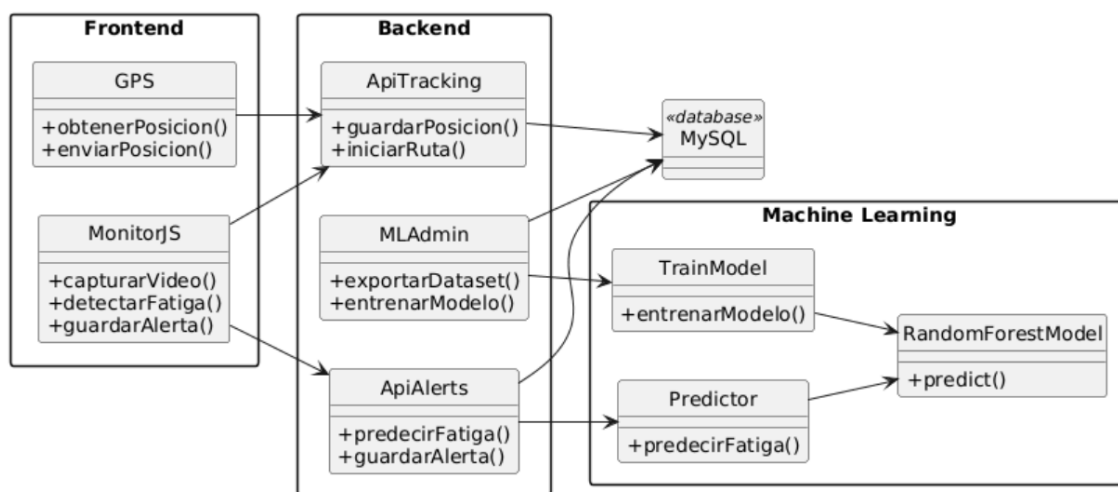
El frontend obtiene las características faciales mediante MediaPipe Face Mesh y las envía al backend. Posteriormente, el backend consulta al servicio de IA para obtener la predicción del nivel de fatiga. Finalmente, la información es almacenada en la base de datos y presentada al usuario mediante alertas visuales y sonoras.

7.1.3 Diagrama de Clases

El sistema está organizado utilizando diferentes clases y módulos especializados, donde cada uno cumple una responsabilidad específica.

Las principales clases consideradas en la arquitectura son (Anexo 1, pag.):

Clase	Responsabilidad
Predictor	Cargar el modelo Random Forest y realizar predicciones de fatiga.
PredictorService	Exponer el servicio REST mediante FastAPI.
ApiAlerts	Procesar las alertas recibidas desde el navegador y solicitar la predicción al modelo.
MLAdmin	Administrar el entrenamiento, exportación del dataset y versionado del modelo.
Monitor	Capturar las características faciales mediante MediaPipe y enviarlas al servidor.
DetectorOpenCV	Detectar rostro y ojos utilizando OpenCV como alternativa de procesamiento en Python.



Estas clases trabajan de manera coordinada para garantizar que las predicciones se realicen en tiempo real y que las muestras generadas puedan reutilizarse posteriormente para mejorar el modelo.

7.1.4 Diagrama de Casos de Uso

Los principales actores identificados dentro del sistema son (Anexo 1, pag.):

Administrador

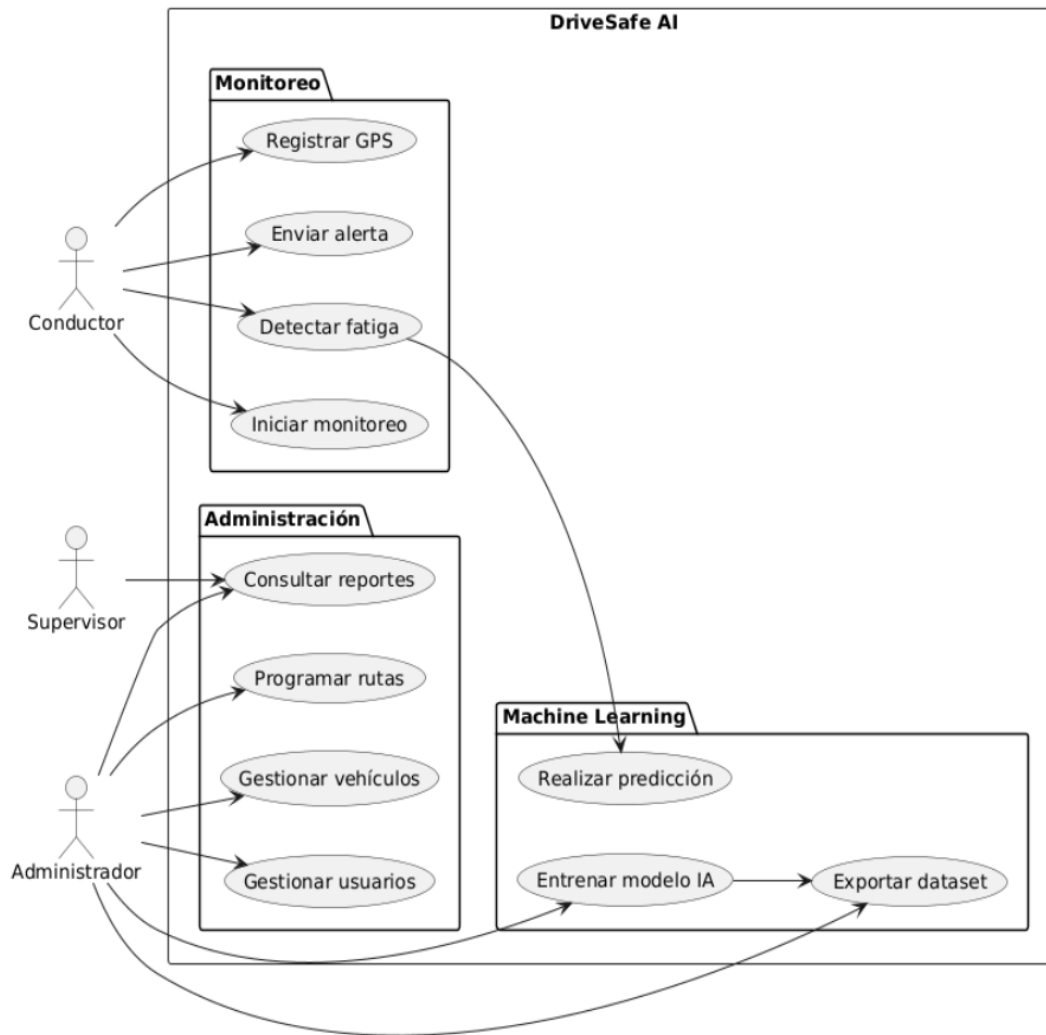
- Gestionar usuarios.
- Gestionar conductores.
- Gestionar vehículos.
- Programar rutas.
- Visualizar alertas.
- Consultar estadísticas.
- Reentrenar el modelo de Machine Learning.
- Supervisar el estado del sistema.

Supervisor

- Monitorear rutas activas.
- Consultar ubicación GPS.
- Visualizar alertas de fatiga.
- Revisar incidentes reportados.

Conductor

- Iniciar sesión.
- Seleccionar ruta asignada.
- Activar el monitoreo.
- Recibir alertas de fatiga.
- Reportar incidentes.
- Finalizar el recorrido.



8. Integración y Consumo del Modelo

El modelo de **Machine Learning** fue integrado al sistema **DriveSafe AI** con el propósito de realizar predicciones de fatiga en tiempo real durante el monitoreo del conductor. La integración permite que los datos obtenidos desde la aplicación web sean enviados al modelo entrenado, el cual devuelve un porcentaje estimado de fatiga que posteriormente es utilizado por el sistema para generar alertas y recomendaciones.

El modelo entrenado se encuentra almacenado en el archivo **modelo_fatiga.pkl**, generado durante el proceso de entrenamiento mediante la librería **Joblib**. Este archivo es cargado automáticamente por el servicio de predicción y reutilizado en cada solicitud, evitando que el modelo tenga que entrenarse nuevamente.

8.1 Exportación del modelo

Una vez finalizado el entrenamiento, el modelo es serializado utilizando **Joblib** para generar el archivo **modelo_fatiga.pkl**, el cual contiene toda la información necesaria para realizar predicciones posteriormente.

La exportación se realiza mediante la siguiente instrucción:

```
joblib.dump(modelo, "modelo_fatiga.pkl")
```

Posteriormente, el archivo es cargado mediante:

```
modelo = joblib.load("modelo_fatiga.pkl")
```

Esta técnica permite reducir el tiempo de procesamiento y facilita el despliegue del sistema en diferentes equipos sin necesidad de volver a entrenar el modelo.

8.2 Consumo del modelo mediante la API de IA

El sistema utiliza un servicio desarrollado en Python con **FastAPI** para realizar las predicciones. Cuando el conductor está siendo monitoreado, el navegador obtiene las características faciales detectadas y las envía al servidor.

Las variables enviadas corresponden a:

- Apertura de ojos (EAR).
- Cantidad de bostezos.
- Tiempo de conducción.

Estas variables son recibidas por el servicio FastAPI mediante el endpoint:

POST /predict

El servicio carga el modelo **modelo_fatiga.pkl**, procesa las variables recibidas y devuelve un objeto JSON con el nivel estimado de fatiga.

Ejemplo de respuesta:

```
{"fatiga": 68}
```

8.3 Integración con la aplicación web

La comunicación entre el sistema web y el modelo de Machine Learning se realiza desde el archivo **api_alerts.php**.

Cuando el sistema recibe las variables capturadas por el navegador, primero intenta comunicarse con el servicio **FastAPI** mediante una petición HTTP al endpoint de predicción.

http://127.0.0.1:8000/predict

Si el servicio responde correctamente, el valor de fatiga obtenido es utilizado para registrar la alerta correspondiente y almacenarla en la base de datos.

En caso de que el servicio **FastAPI** no se encuentre disponible, el sistema implementa un mecanismo de respaldo (**fallback**), ejecutando directamente el script **predictor.py**, garantizando que la predicción pueda realizarse incluso cuando el servicio web no esté activo.

Este mecanismo incrementa la disponibilidad del sistema y evita la interrupción del proceso de monitoreo.

8.4 Flujo de integración del modelo

El funcionamiento completo del proceso se resume en las siguientes etapas:

- El navegador captura las características faciales del conductor mediante **MediaPipe Face Mesh**.
- Se calculan las variables de entrada: apertura de ojos, cantidad de bostezos y tiempo de conducción.
- Las variables son enviadas **a api_alerts.php**.
- El sistema consulta el servicio **FastAPI** utilizando el modelo **modelo_fatiga.pkl**.
- El modelo devuelve el porcentaje estimado de fatiga.
- La aplicación interpreta el resultado y determina el nivel de riesgo.

- Finalmente se registra la alerta en la base de datos y se presenta la recomendación correspondiente al conductor.

9. Visualización de Resultados y Lógica de Negocio

Una vez obtenida la predicción del modelo de **Machine Learning**, el sistema **DriveSafe AI** interpreta el nivel de fatiga del conductor y ejecuta la lógica de negocio correspondiente. El objetivo es transformar el resultado numérico generado por el modelo en información comprensible para el usuario, permitiéndole tomar decisiones oportunas durante la conducción.

El valor de fatiga devuelto por el modelo corresponde a un porcentaje entre 0 y 100. Posteriormente, el sistema clasifica dicho valor en cuatro niveles de riesgo:

Nivel de fatiga	Rango
Bajo	0 – 23
Medio	24 – 47
Alto	48 – 71
Crítico	72 – 100

Esta clasificación permite que el sistema determine automáticamente las acciones que deben ejecutarse según el estado del conductor.

9.1 Lógica de negocio

Durante el monitoreo, el navegador obtiene continuamente las características faciales del conductor mediante **MediaPipe Face Mesh**. Estas características son enviadas al modelo de **Machine Learning**, el cual calcula el porcentaje de fatiga.

Con el resultado obtenido, el sistema realiza las siguientes acciones:

- Registrar la alerta en la base de datos.
- Clasificar el nivel de riesgo.
- Mostrar el estado de fatiga en la interfaz.
- Generar recomendaciones para el conductor.
- Guardar las características utilizadas para futuros reentrenamientos del modelo.

- Registrar la ubicación GPS asociada a la alerta.

De esta manera, la predicción realizada por la Inteligencia Artificial no solamente informa un valor numérico, sino que activa diferentes procesos del sistema para mejorar la seguridad durante la conducción.

9.2 Visualización de resultados

La interfaz web muestra al conductor el nivel de fatiga obtenido mediante el modelo de **Machine Learning**. Dependiendo del porcentaje calculado, el sistema presenta mensajes de advertencia y recomendaciones que permiten al usuario conocer su estado durante el recorrido.

Entre los elementos visualizados se encuentran:

- Nivel de fatiga.
- Tiempo de conducción.
- Cantidad de bostezos detectados.
- Estado del monitoreo.
- Alertas generadas.
- Información de la ruta.
- Posición GPS durante el recorrido.

Cuando el nivel de fatiga aumenta, la interfaz actualiza automáticamente la información y presenta advertencias más visibles para el conductor.

9.3 Casos de prueba

Se realizaron pruebas utilizando diferentes valores de entrada para verificar el funcionamiento del modelo de Machine Learning y su integración con el sistema.

Caso de prueba 1

Datos de entrada

- Apertura de ojos (EAR): 0.34
- Bostezos: 0
- Tiempo de conducción: 10 minutos

Resultado obtenido

- Fatiga predicha: 15
- Nivel: Bajo
- Respuesta del sistema

El sistema mantiene el monitoreo sin generar alertas críticas y registra la información en la base de datos.

Caso de prueba 2

Datos de entrada

- Apertura de ojos (EAR): 0.23
- Bostezos: 2
- Tiempo de conducción: 40 minutos

Resultado obtenido

- Fatiga predicha: 56
- Nivel: Alto

Respuesta del sistema

Se registra una alerta de fatiga alta, se almacena la información en la tabla **ml_samples** y se recomienda al conductor realizar una pausa para descansar.

Caso de prueba 3

Datos de entrada

- Apertura de ojos (EAR): 0.14
- Bostezos: 4
- Tiempo de conducción: 70 minutos

Resultado obtenido

- Fatiga predicha: 88
- Nivel: Crítico

Respuesta del sistema

El sistema genera una alerta crítica, registra la evidencia correspondiente, almacena las características faciales para futuros entrenamientos y muestra una recomendación inmediata indicando que el conductor debe detener el vehículo y descansar antes de continuar el viaje.

9.4 Flujo general de funcionamiento

El funcionamiento completo del sistema puede resumirse en las siguientes etapas:

- El conductor inicia el monitoreo desde la aplicación web.
- **MediaPipe Face Mesh** detecta continuamente los puntos faciales, pero no se ven a simple vista.
- Se calculan las variables de entrada (features) que son: apertura de ojos, cantidad de bostezos y tiempo de conducción.
- Las variables son enviadas al servicio de predicción.
- El modelo Random Forest estima el nivel de fatiga.
- El sistema interpreta el resultado obtenido.
- Se generan alertas y recomendaciones según el nivel detectado.
- Finalmente, toda la información es almacenada en la base de datos para permitir el seguimiento del conductor y el futuro reentrenamiento del modelo.

10. Conclusion

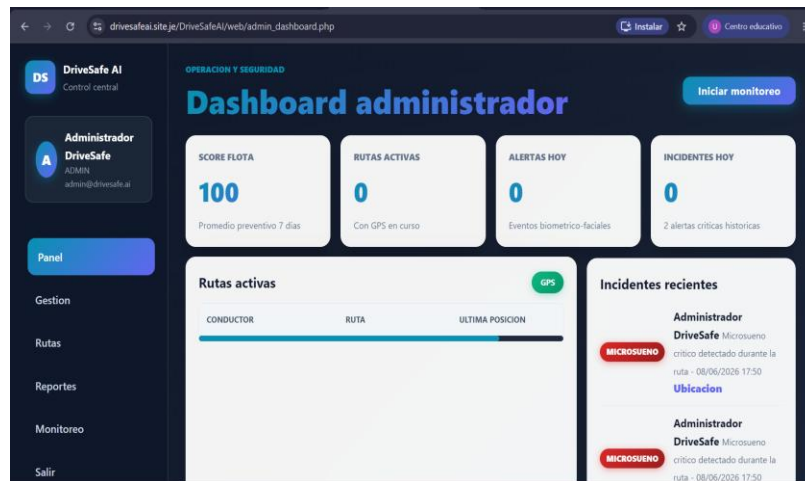
El proyecto **DriveSafe AI** logró integrar técnicas de aprendizaje automático dentro de una aplicación web para la detección de fatiga en conductores. El sistema combina procesamiento de imágenes, monitoreo en tiempo real y un modelo de Machine Learning basado en Random Forest para estimar el nivel de fatiga a partir de características faciales como la apertura de los ojos, el número de bostezos y el tiempo de conducción.

La arquitectura implementada permitió desacoplar el servicio de inteligencia artificial mediante **FastAPI**, facilitando el mantenimiento, la actualización del modelo y la integración con el sistema desarrollado en PHP. Además, la recolección continua de nuevas muestras posibilita el reentrenamiento del modelo, favoreciendo una mejora progresiva en la precisión de las predicciones.

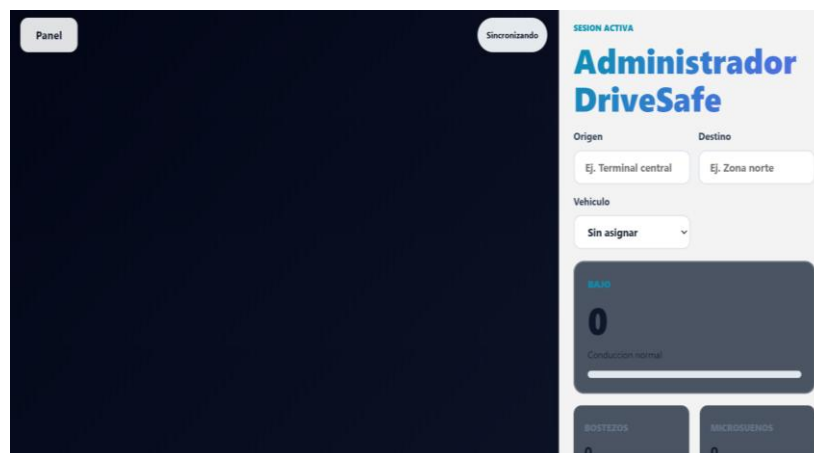
Finalmente, la solución desarrollada demuestra que es posible integrar técnicas de Inteligencia Artificial en aplicaciones web reales para apoyar la toma de decisiones y contribuir a la prevención de accidentes ocasionados por fatiga del conductor.

11. Anexos

Dashboard del administrador



Monitor sin iniciar el monitoreo, ni datos de la ruta y vehiculo



Monitoreo iniciado, con una alerta de somnolencia



12. Partes del código web relacionadas con ML:

1. Train_model (entrenamiento del ML con Random Forest Regressor)

```
import pandas as pd
from sklearn.ensemble import RandomForestRegressor
import joblib

df = pd.read_csv("dataset.csv")

X = df[["ojos", "bostezos", "tiempo"]]
y = df["fatiga"]

modelo = RandomForestRegressor()
modelo.fit(X, y)

joblib.dump(modelo, "modelo_fatiga.pkl")

print("Modelo listo")
```

predictor.py (funciona sin conexión a una API)

```
import joblib
import numpy as np
import sys
import os

# Cargar modelo relativo al directorio del script (funciona aunque el
proceso se ejecute desde otra carpeta)
MODEL_PATH = os.path.join(os.path.dirname(__file__), "modelo_fatiga.pkl")
modelo = joblib.load(MODEL_PATH)

def predecir_fatiga(ojos, bostezos, tiempo):
    datos = np.array([[ojos, bostezos, tiempo]])
    return float(modelo.predict(datos)[0])

if __name__ == "__main__":
    # Permite invocar desde la línea de comandos: python predictor.py <ojos>
    <bostezos> <tiempo>
    if len(sys.argv) >= 4:
        try:
            ojos = float(sys.argv[1])
            bostezos = float(sys.argv[2])
            tiempo = float(sys.argv[3])
            print(predecir_fatiga(ojos, bostezos, tiempo))
        except Exception as e:
```

```
        print("0.0")
    else:
        print("0.0")
```

2. web/api_alerts.php (Líneas 33-84) - Predicción ML

```
// Calcular fatiga: preferir fatiga enviada por el cliente; si no existe,
usar el predictor Python si se reciben features-> variables de entrada
$fatiga = null;
if (isset($payload["fatiga"]) && $payload["fatiga"] !== null) {
    $fatiga = max(0, min(100, (int) round((float) $payload["fatiga"])));
} elseif (isset($payload["ojos"]) && isset($payload["bostezos"]) &&
isset($payload["tiempo"])) {
    $ojos = (float) $payload["ojos"];
    $bostezos = (float) $payload["bostezos"];
    $tiempo = (float) $payload["tiempo"];

    // Intentar llamar al microservicio FastAPI en localhost:8000/predict
    $predict_ok = false;
    $pred_value = 0.0;

    if (function_exists('curl_init')) {
        $ch = curl_init('http://127.0.0.1:8000/predict');
        curl_setopt($ch, CURLOPT_POST, true);
        curl_setopt($ch, CURLOPT_RETURNTRANSFER, true);
        curl_setopt($ch, CURLOPT_CONNECTTIMEOUT, 1);
        curl_setopt($ch, CURLOPT_TIMEOUT, 2);
        curl_setopt($ch, CURLOPT_HTTPHEADER, ["Content-Type:
application/json"]);
        $body = json_encode(["ojos" => $ojos, "bostezos" => $bostezos,
"tiempo" => $tiempo]);
        curl_setopt($ch, CURLOPT_POSTFIELDS, $body);
        $res = curl_exec($ch);
        $err = curl_errno($ch);
        curl_close($ch);

        if ($res !== false && !$err) {
            $decoded = json_decode($res, true);
            if (is_array($decoded) && isset($decoded['fatiga'])) {
                $pred_value = (float) $decoded['fatiga'];
                $predict_ok = true;
            }
        }
    }
}
```

```

    // Fallback: si el servicio no responde, usar el predictor.py por
    shell_exec
    if (!$predict_ok) {
        $py = 'python';
        $cmd = $py . ' ' . escapeshellarg(__DIR__ . '/../predictor.py') . '
' . escapeshellarg((string)$ojos) . ' ' . escapeshellarg((string)$bostezos)
. ' ' . escapeshellarg((string)$tiempo);
        $out = null;
        try {
            $out = shell_exec($cmd);
            $pred_value = is_string($out) ? floatval(trim($out)) : 0.0;
        } catch (Exception $e) {
            $pred_value = 0.0;
        }
    }

    $fatiga = max(0, min(100, (int) round($pred_value)));
} else {
    $fatiga = 0;
}

```

web/api_alerts.php (Líneas 132-157) - Guardar muestras ML

```

// Guardar muestra para ML si el cliente envió features
(ojos/bostezos/tiempo)
if (isset($payload["ojos"]) || isset($payload["bostezos"]) ||
isset($payload["tiempo"])) {
    // Asegurar la tabla existe
    $conexion->query("CREATE TABLE IF NOT EXISTS ml_samples (
        id INT AUTO_INCREMENT PRIMARY KEY,
        id_alerta INT NULL,
        id_conductor INT NULL,
        created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
        ojos DOUBLE NULL,
        bostezos INT NULL,
        tiempo DOUBLE NULL,
        fatiga INT NULL
    ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4");

    $ojosVal = isset($payload["ojos"]) ? (float) $payload["ojos"] : null;
    $bostVal = isset($payload["bostezos"]) ? (int) $payload["bostezos"] :
null;
    $tiempoVal = isset($payload["tiempo"]) ? (float) $payload["tiempo"] :
null;
    $fatigaVal = $fatiga !== null ? (int) $fatiga : null;
}

```

```

        $ins = $conexion->prepare("INSERT INTO ml_samples (id_alerta,
id_conductor, ojos, bostezos, tiempo, fatiga) VALUES (?, ?, ?, ?, ?, ?)");
        if ($ins) {
            // tipos: id_alerta (i), id_conductor (i), ojos (d), bostezos (i),
            tiempo (d), fatiga (i)
            $ins->bind_param("iididi", $idAlerta, $idConductor, $ojosVal,
$bostVal, $tiempoVal, $fatigaVal);
            $ins->execute();
            $ins->close();
        }
    }
}

```

3. web/ml_admin.php (Líneas 9-55) - Entrenamiento del modelo

```

if ($_SERVER["REQUEST_METHOD"] === "POST" && isset($_POST['action']) &&
$_POST['action'] === 'train') {
    // exporta dataset a CSV y corre train_model.py

    $outFile = __DIR__ . '/../dataset.csv';

    $rows = [];
    $res = $conexion->query("SELECT ojos, bostezos, tiempo, fatiga FROM
ml_samples WHERE ojos IS NOT NULL AND bostezos IS NOT NULL AND tiempo IS NOT
NULL AND fatiga IS NOT NULL");
    if ($res) {
        while ($r = $res->fetch_assoc()) {
            $rows[] = $r;
        }
        $res->close();
    }

    if (count($rows) === 0) {
        $error = "No hay muestras ML completas para exportar. Recolecta más
eventos con features (ojos, bostezos, tiempo).";
    } else {
        // escribe un CSV con las muestras de train_model.py

        $fp = fopen($outFile, 'w');
        if ($fp === false) {
            $error = "No se pudo crear el archivo dataset.csv en la raíz del
proyecto (permisos).";
        } else {
            fputcsv($fp, ['ojos', 'bostezos', 'tiempo', 'fatiga']);
            foreach ($rows as $r) {

```

```

        fputcsv($fp, [$r['ojos'], $r['bostezos'], $r['tiempo'],
$r['fatiga']]);
    }
    fclose($fp);

    // existe modelo previo? hacer backup

    $modelPath = __DIR__ . '/../modelo_fatiga.pkl';
    if (file_exists($modelPath)) {
        $bak = __DIR__ . '/../modelo_fatiga_' . date('Ymd_His') .
'.pkl';
        copy($modelPath, $bak);
    }

    // corre train_model.py

    $cmd = 'cd ' . escapeshellarg(__DIR__ . '/../') . ' && python ' .
escapeshellarg('train_model.py') . ' 2>&1';
    $out = shell_exec($cmd);

    // modelo entrenado, guardar versión

    $verFile = __DIR__ . '/../model_version.txt';
    $verContent = "trained_at=" . date('c') . "\noutput=\n" . ($out
?? '') . "\n";
    file_put_contents($verFile, $verContent);

    $message = "Entrenamiento ejecutado. Salida:\n" . ($out ?? '(sin
salida)');
    }
}
}

```

4. web/ml_admin.php (Líneas 58-78) - Métricas del dataset

```

// Estados de dataset y modelo
$totalSamples = (int) db_value($conexion, "SELECT COUNT(*) FROM ml_samples",
"", []);
$totalWithFeatures = (int) db_value($conexion, "SELECT COUNT(*) FROM
ml_samples WHERE ojos IS NOT NULL AND bostezos IS NOT NULL AND tiempo IS NOT
NULL AND fatiga IS NOT NULL", "", []);

// Métricas adicionales del dataset
$avgFatiga = db_value($conexion, "SELECT AVG(fatiga) FROM ml_samples WHERE
fatiga IS NOT NULL", "", []);

```

```

$maxFatiga = db_value($conexion, "SELECT MAX(fatiga) FROM ml_samples WHERE
fatiga IS NOT NULL", "", []);
$minFatiga = db_value($conexion, "SELECT MIN(fatiga) FROM ml_samples WHERE
fatiga IS NOT NULL", "", []);
$avgOjos = db_value($conexion, "SELECT AVG(ojos) FROM ml_samples WHERE ojos
IS NOT NULL", "", []);
$avgBostezos = db_value($conexion, "SELECT AVG(bostezos) FROM ml_samples
WHERE bostezos IS NOT NULL", "", []);
$avgTiempo = db_value($conexion, "SELECT AVG(tiempo) FROM ml_samples WHERE
tiempo IS NOT NULL", "", []);

// Distribución de niveles de fatiga
$distribucion = [
    'BAJO' => (int) db_value($conexion, "SELECT COUNT(*) FROM ml_samples
WHERE fatiga < 24", "", []),
    'MEDIO' => (int) db_value($conexion, "SELECT COUNT(*) FROM ml_samples
WHERE fatiga >= 24 AND fatiga < 48", "", []),
    'ALTO' => (int) db_value($conexion, "SELECT COUNT(*) FROM ml_samples
WHERE fatiga >= 48 AND fatiga < 72", "", []),
    'CRITICO' => (int) db_value($conexion, "SELECT COUNT(*) FROM ml_samples
WHERE fatiga >= 72", "", [])
];

$modelVersion = file_exists(__DIR__ . '/../model_version.txt') ?
file_get_contents(__DIR__ . '/../model_version.txt') : null;
$modelExists = file_exists(__DIR__ . '/../modelo_fatiga.pkl');

```

5. web/assets/monitor.js (Líneas 359-390) - Envío de features ML

```

// Guardar alerta en el servidor o en la cola offline
async function saveAlert(evento, nivel, fatiga, recomendacion, features =
{}) {
    const clientAlertId = `alert-${Date.now()}-${
    ${Math.random().toString(16).slice(2)}}`;
    const fechaEvento = new Date().toISOString();
    // enviar también las features usadas por el cliente para ML
    const payload = {
        evento,
        nivel,
        fatiga,
        recomendacion,
        id_viaje: routeId,
        latitud: lastPosition?.latitud ?? null,
        longitud: lastPosition?.longitud ?? null,
    };
}

```

```

        fecha_evento: fechaEvento,
        clientAlertId,
        // Features para entrenamiento/predicción
        ojos: typeof features.ojos !== 'undefined' ? features.ojos : (typeof
lastEar !== 'undefined' && lastEar !== null ? lastEar : null),
        bostezos: typeof features.bostezos !== 'undefined' ?
features.bostezos : (typeof yawnCount !== 'undefined' ? yawnCount : null),
        tiempo: typeof features.tiempo !== 'undefined' ? features.tiempo :
(sessionStartedAt ? Math.floor((Date.now() - sessionStartedAt) / 60000) : 0)
    };

    try {
        const result = await postJson("api_alerts.php", payload);
        if (result?.id_alerta) {
            await setLocalMapping(clientAlertId, result.id_alerta);
        }
        return { ...result, clientAlertId };
    } catch (error) {
        console.warn("No se pudo registrar alerta", error);
        await addOfflineJob("alert", payload);
        return { ok: false, offline: true, clientAlertId };
    }
}

```

6. web/assets/monitor.js (Líneas 646, 655, 676, 775) - Llamadas con features ML (datos de ingreso)

```

// llamadas con evidencia para microsueño critico
    saveAlertWithEvidence("MICROSUEÑO", "CRITICO", 90,
getRecommendation("CRITICO"), true, {ojos: ear, bostezos: yawnCount, tiempo:
minutes});

```

```

// Guardar alerta de bostezo con evidencia
    yawnCountEl.textContent = yawnCount;

```

```

// Guardar alerta con evidencia si ha pasado suficiente tiempo desde la
última alerta alta o critica
    if (now - lastHighAlertAt > 20000) {
        lastHighAlertAt = now;
        saveAlertWithEvidence(level === "CRITICO" ? "FATIGA_CRITICA" :
"FATIGA_ALTA", level, fatigue, recommendation, false, {ojos: ear, bostezos:
yawnCount, tiempo: minutes});
    }

```

```
// Botón de reporte de incidente manual
incidentButton.addEventListener("click", () => {
    reportIncident("ACCIDENTE", "Accidente reportado manualmente por el conductor");
    saveAlertWithEvidence("FATIGA_CRITICA", "CRITICO", 100, "Accidente reportado. Verificar ubicacion inmediatamente", true, {ojos: null, bostezos: yawnCount, tiempo: sessionStartedAt ? Math.floor((Date.now() - sessionStartedAt) / 60000) : 0});
});
```

Flujo ML funcional:

1. Predicción ML

- **monitor.php/monitor.js:** Detecta características faciales (ojos, bostezos, tiempo)
- **api_alerts.php:** Usa el modelo entrenado (**modelo_fatiga.pkl**) para predecir fatiga
- **Prioridad:** Servicio FastAPI (<http://127.0.0.1:8000/predict>)
- **Fallback:** predictor.py si el servicio no está activo

2. Acumulación de datos

- Cada alerta guarda las características en tabla **ml_samples**
- **Datos:** ojos, bostezos, tiempo, fatiga

3. Entrenamiento

- **ml_admin.php:** Panel de administración ML
- Exporta **ml_samples** → **dataset.csv**
- Ejecuta **train_model.py** → entrena Random Forest
- Genera nuevo **modelo_fatiga.pkl**
- Backup automático del modelo anterior

4. Scripts funcionales

- **start_ml_service.bat** - Inicia servicio **FastAPI**
- **start_detector.bat** - Inicia detector con cámara (**detector_opencv.py**)
- **install_requirements.bat** - Instala dependencia

5. Link de la pagina subida a un hosting gratuito

<https://drivesafeai.site.je/DriveSafeAI/web/login.php>

Datos creados para ingresar:

Administrador: admin@drivesafe.ai

Conductor demo: conductor@drivesafe.ai

Contraseña(ambos): admin123

Repositorio Github: <https://github.com/nmusnayo/DriveSafeAI.git>